

Cursor Hunter Prototype V0.0.0.1

이 문서는 Cursor Hunter의 현재 프로토타입 구현 상태와 다음 확장 방향을 공유하기 위한 기술 인수인계 문서다. 새로 합류한 개발자는 이 문서를 먼저 읽고, 그 다음 실제 코드와 씬을 확인한다.

- [가독성 개선 PDF](#)

- 기준일: 2026-09-14

- Unity: 6000.3.13f1

- 런타임 진입점: [Assets/CursorHunter/App/Scenes/Main.unity](#)

- 현재 단계: 일반 필드 전투 세로 슬라이스 + 런타임 화면 라우팅 1차

- 문서 성격: 현재 구현을 기준으로 한 사실 문서. 미구현 기능은 계획으로 구분한다.

1. 이 프로토타입의 방향

Cursor Hunter는 화면 위 몬스터에 커서를 겹친 뒤 좌클릭으로 공격하고, 짧은 런에서 얻은 성장 재화로 다음 런의 공격력을 확장하는 커서 액션 성장 게임이다.

장기 게임 루프는 다음과 같다.

```
메인 메뉴
→ 일반 필드 60초
→ 런 정산
→ 특성 구매
→ 일반 필드 반복
→ 보스 필드
→ 보스 파훼 성공/실패
→ 해금·정산·저장
→ 다음 보스와 엔딩
```

V0.0.0.1은 이 전체 루프를 완성한 버전이 아니다. 현재는 다음 질문에 답하는 데 목적이 있다.

1. 하나의 런이 시작·진행·종료될 수 있는가?
2. 현재 런에 속한 대상만 공격되는가?
3. 스폰 또는 준비 실패가 부분 실행으로 남지 않는가?
4. 씬의 UI 화면 전환이 개별 버튼의 `SetActive` 호출에 흩어지지 않는가?
5. 이후 여러 몬스터·보스·보상·특성으로 확장할 경계가 보이는가?

전체 기획 수치와 화면 설계는 [docs/cursor-hunter-game-design-v0.0.4.md](#)를 기준으로 한다. 단, 기획 문서에 있는 기능이 이 프로토타입에 이미 구현되었다는 뜻은 아니다.

2. 현재 구현 상태

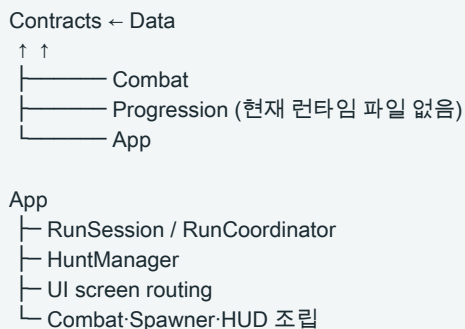
영역	상태	현재 범위
App 진입점	구현	HuntManager가 런 요청을 만들고 Combat과 UI를 연결한다.
런 상태 머신	구현	RunSession의 Idle/Starting/Running/Paused/Completing/Completed/Aborting/Aborted
런 계약	구현	RunId, seed, schema/balance version, RunRequest, RunResult
원자적 시작	구현	Combat 준비 → 초기 스폰 → Combat 커밋 → Session 커밋, 실패 시 rollback
일반 전투	구현	커서 위치 갱신, overlap 대상 수집, 쿨다운, 피해·처치·가넷 사실 집계
대상 소속 검증	1차 구현	RunId, IsActive, IsRegistered 검증. 별도 registry는 아직 없음
스폰	1종 프로토타입	SpawnPlan은 목록 구조지만 MonsterSpawner는 현재 단일 entry만 소비
몬스터 데이터	1종 구현	SlimeDefinition + 기존 Walker_Stump.prefab
ID 기반 경계	부분 구현	monsterId, prefabKey, SpawnSnapshot은 존재하지만 Catalog/Registry는 미구현
UI 화면 라우팅	1차 구현	AppUiRootController가 화면 root의 단일 활성 상태를 관리한다.
전투 HUD	프로토타입	PrototypeRunHud가 런타임에 Legacy Text와 결과 패널을 만든다.
Progression	미구현	asmdef와 폴더 경계만 존재하며 구매·프로필·저장은 연결되지 않았다.
정산·저장	미구현	RunResult는 생성되지만 지갑·저장·중복 지급 방지는 아직 없다.
보스·파훼·스킬·자동 공격	미구현	계약과 기획만 있고 현재 런타임에는 연결되지 않았다.
풀링·대량 대상 최적화	미구현	현재는 Instantiate/Destroy와 단일 리스트를 사용한다.

현재 씬의 프로토타입 수치는 기획의 최종 수치와 다르다.

- durationSeconds: 15
- monsterId: monster.slime
- maxHealth: 30
- spawnIntervalSeconds: 1.5
- packSize: 1
- garnetReward: 3
- schemaVersion: 1
- balanceVersion: 1
- allowPrototypeFallback: false

따라서 이 씬은 “15초 동안 동작하는 일반 필드 기술 검증”으로 이해해야 한다. 최종 60초 밸런스가 적용된 빌드로 취급하지 않는다.

3. 런타임 모듈 구조



실제 asmdef 참조 방향은 다음과 같다.

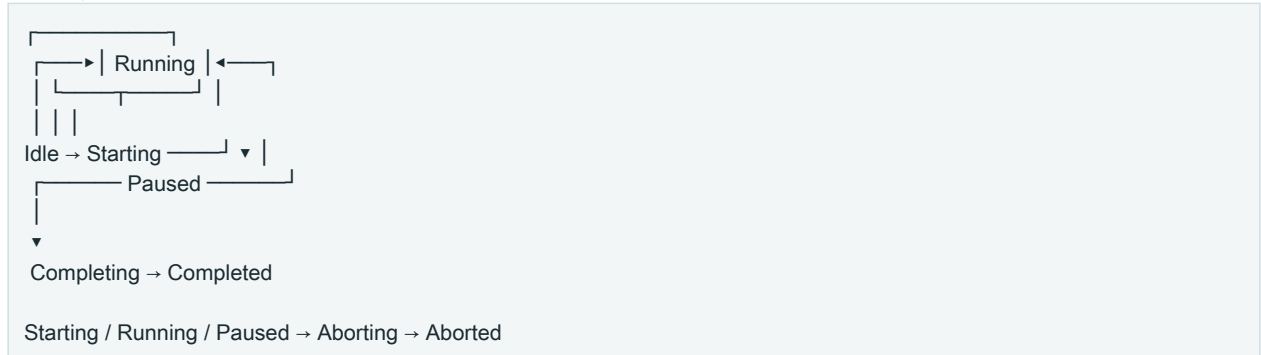
어셈블리	참조	책임
CursorHunter.Contracts	없음, UnityEngine 비의존	런 입출력과 값 계약
CursorHunter.Data	Contracts	ScriptableObject 정적 정의
CursorHunter.Combat	Contracts	전투, 대상, 스폰, 랜덤 위치
CursorHunter.Progression	Contracts, Data	현재는 빈 확장 경계
CursorHunter.App	Contracts, Data, Progression, Combat	진입점과 모듈 조립
CursorHunter.App.Tests	App, Combat, Contracts, Data	EditMode 테스트

원칙은 다음과 같다.

- Contracts에는 **GameObject**, **ScriptableObject**, 저장 경로를 넣지 않는다.
- Combat은 지갑·프로필·파일 저장을 직접 변경하지 않는다.
- App이 Data/Progression의 입력을 스냅샷으로 만들어 Combat에 전달한다.
- 전투 프리팹의 외형과 런타임 HP·보상은 분리한다.
- 공용 전역 이벤트 버스, DI 프레임워크, Addressables는 현재 단계에 도입하지 않는다.
- 공급사 스크립트와 기존 원본 에셋은 직접 재구성하지 않고 게임 전용 래퍼/Variant 경계로 감싼다.

4. 런 생명주기

4.1 상태 전이



Completing과 Aborting은 현재 즉시 terminal state로 넘어가는 전이 상태다. 외부에서 terminal state를 재시작하거나 Complete와 Abort를 모두 성공시킬 수 없다.

4.2 시작 흐름

HuntManager는 다음 순서로 하나의 런을 만든다.

1. Combat, Spawner, UI 참조를 확인한다.
2. MonsterDefinition에서 SpawnSnapshot과 SpawnPlan을 만든다.
3. RunId.Create()로 고유 ID를 만들고 seed, schema version, balance version, mode, duration을 RunRequest에 담는다.
4. RunCoordinator가 RunId를 claim하고 RunSession.Start()를 호출한다.
5. CombatRunController.PrepareRun()으로 공격 시계를 열지 않고 전투 데이터를 준비한다.
6. MonsterSpawner.StartRun()이 초기 spawn pack을 만든다.
7. Combat을 commit한 뒤 RunSession.CommitStart()로 Running에 진입한다.
8. 시작이 성공한 경우에만 UI를 Combat 화면으로 전환하고 커서와 HUD를 켜다.

어느 단계에서든 실패하면 다음을 수행한다.

스폰/준비 실패
→ 이미 생성한 대상 unregister + destroy

→ Combat prepared 상태 취소 또는 abort
 → RunSession.Abort(StartFailed, Discard)
 → UI는 기존 선택 화면에 유지

필드 시작 버튼에 연결된 추가 SetActive(false) 이벤트는 제거되었다. 따라서 스폰 실패가 화면을 먼저 숨기는 부작용을 만들지 않는다.

4.3 종료 규칙

호출	Session 결과	정산 의미
시간 만료	Completed	Eligible 결과 생성
EndPrototypeRun()	Aborted	현재 프로토타입에서는 Eligible 결과 표시 가능
ResetPrototypeRun()	Aborted	Reset + Discard, 완료 이벤트 없음
ReturnToMainMenu()	Aborted 후 메인 메뉴	스폰·커서·HUD·테스트 패널 정리
시작 실패	Aborted	StartFailed + Discard
수치 overflow	Aborted	NumericOverflow + Discard

동일 RunId는 하나의 RunCoordinator lifetime에서 새 전투 시작에 재사용할 수 없다. 향후 저장 정산 재시도는 새 전투가 아니므로 같은 ID를 유지하는 별도 경로로 둔다.

5. Combat 현재 구조

5.1 입력과 공격

MainCursorController
 ↳ 커서 Transform 이동, native cursor 표시/숨김, 범위 배율

CursorAttackController
 ↳ 좌클릭 감지 → UI pointer 차단 확인 → 위치 갱신
 → CombatRunController.TryAttack()

CombatRunController
 ↳ overlap buffer → 대상 중복 제거 → RunId 검증
 → 피해 적용 → 유효 피해/처치/가넷 집계

- 이동만으로 공격하지 않는다.
- 일반 입력은 UI 위에서 공격하지 않는다.
- 개발용 TestAttack()은 테스트 패널 버튼에서 UI pointer 차단을 우회한다.
- CombatSnapshot은 공격력, 범위 배율, 쿨다운, 묶음당 타격 수를 런 시작 시 고정한다.
- $t \geq \text{duration}$ 인 입력은 공격하지 않는다.
- 유효 피해와 가넷 누적은 long overflow를 검사한다.

5.2 현재 런 대상 보호

현재 1차 방어는 대상 컴포넌트의 다음 조건이다.

```
target.RunId == activeRun.RunId
target.IsActive
target.IsRegistered
```

CombatRunController는 overlap collider에서 GetComponentInParent<WalkerStumpTarget>()를 찾는다. 즉, 오래된 런 대상이 물리적으로 남아 있어도 새 런의 피해를 받지 않는다.

다만 WalkerStumpTarget이라는 타입명이 Combat 코드에 남아 있고, 별도 ActiveTargetRegistry는 아직 없다. 여러 몬스터와 풀링을 도입할 때는 다음 순서로 바꾼다.

1. RunId token 검증은 유지한다.
2. Spawner가 생성 시 ActiveTargetRegistry에 등록한다.

3. 사망·반환·런 종료 전에 등록을 해제한다.
4. Combat은 registry에서 현재 런의 ICombatTarget만 조회한다.
5. token은 오래된 Animator 이벤트와 비동기 콜백 방어용으로 계속 유지한다.

5.3 스폰

MonsterSpawner는 현재 다음을 담당한다.

- seed 기반 위치 선택
- 화면 viewport padding 적용
- alive limit과 pack size 적용
- 초기 pack 및 주기적 pack 생성
- 위치/Instantiate/Target 초기화 실패 결과 반환
- 일부만 생성된 pack의 역순 rollback
- 런 종료 시 unregister 후 대상 제거

SpawnPlan은 IReadOnlyList<SpawnPlanEntry>를 복사해 보관하므로 여러 종류로 확장할 자리는 있다. 하지만 현재 Spawner는 Entries.Count == 1만 허용한다. 다음 단계에서 종별 주기, 종별 상한, 라운드로빈 생성 정책을 추가한다.

5.4 대상 어댑터와 애니메이션

WalkerStumpTarget은 이름은 레거시지만 현재는 시각 프리팹을 전투 계약에 연결하는 어댑터다.

- root 또는 자식의 Animator를 자동 탐색한다.
- Hit, Hurt, Damage, hit 상태 후보를 순서대로 찾는다.
- Dead, Death, dead, death 상태 후보를 찾는다.
- Animator 파라미터가 없어도 상태 이름 기반으로 동작한다.
- Collider가 없으면 SpriteRenderer bounds 기반 BoxCollider2D를 생성한다.
- 죽는 즉시 IsRegistered = false가 되어 추가 피해를 차단한다.
- 지연된 사망 애니메이션 callback은 _deathRunId를 확인한다.

원본 몬스터 에셋의 Hit.anim 17개는 현재 Loop가 false로 통일되어 있다. 이는 시각 설정 변경이며, 피해 적용 시점은 애니메이션 이벤트에 의존하지 않는다.

6. Data와 ID 경계

현재 정적 몬스터 정의는 MonsterDefinition 하나다.

```

MonsterDefinition
├─ monsterId
├─ displayName
├─ prefabKey
├─ prefab
├─ maxHealth
├─ spawnIntervalSeconds
├─ packSize
└─ garnetReward
  
```

현재 asset:

- 경로: Assets/CursorHunter/Data/Resources/MonsterDefinitions/SlimeDefinition.asset
- monsterId: monster.slime
- prefabKey: walker_stump
- prefab: 기존 Assets/DownloadAssets/.../Walker/Walker_Stump.prefab

현재는 `prefabKey`를 보유하지만 `PrefabRegistry`로 조회하지 않고 `MonsterDefinition.prefab` 직접 참조를 사용한다. 따라서 다음 컴포넌트는 아직 계획 상태다.

```
MonsterCatalog
PrefabRegistry
RewardTableRegistry
```

새 몬스터를 추가할 때 당장은 `MonsterDefinition` 하나와 전투 어댑터가 필요하다. 이후 registry 단계에서는 Spawner가 특정 `Walker_Stump` 이름이나 raw asset path를 몰라야 한다.

7. UI 현재 구조

7.1 화면 root

Main.unity의 UICanvas 아래 base screen은 다음과 같이 관리한다.

```
UICanvas
├─ Group_Buttons → MainMenu
├─ FieldPanel_Root → FieldSelect
├─ BossPanel_Root → BossSelect
├─ TraitPanel_Root → Trait
├─ SettingPanel_Root → Settings
├─ InGameObjects → Combat
└─ TestPanel → 개발용 overlay
```

썸 시작 시 `Group_Buttons`만 활성화하고, 선택 패널과 `InGameObjects`는 비활성화한다. `TestPanel`은 Space 입력 또는 기존 테스트 버튼으로 별도 표시한다.

7.2 AppUiRootController

[AppUiRootController](#)는 화면 ID와 root GameObject의 serialized binding을 소유한다.

공개 진입점은 다음과 같다.

- ShowMainMenu()
- ShowFieldSelect()
- ShowBossSelect()
- ShowTrait()
- ShowSettings()
- EnterCombat()
- CloseFieldSelect() / CloseBossSelect() / CloseTrait() / CloseSettings()

개별 Button은 root에 직접 `SetActive`를 호출하지 않고 이 메서드를 호출한다. 이를 통해 화면 root가 늘어나도 전환 규칙은 한 곳에 남는다. `ScreenChanged` 이벤트는 향후 overlay나 화면별 presenter 연결에 사용할 수 있다.

기존 [RootUI_Controller](#)는 사용자가 만든 기존 컴포넌트이므로 삭제하지 않고 호환용으로 남아 있다. 실제 화면 전환 권한은 `AppUiRootController`에 있다.

7.3 현재 HUD의 한계

[PrototypeRunHud](#)는 현재 다음 방식이다.

- UICanvas를 찾는다.
- `UnityEngine.UI.Text`를 런타임에 만든다.
- 매 프레임 TIME/KILLS/GARNET 문자열을 갱신한다.
- 결과 패널도 런타임에 생성한다.

이는 빠른 기술 검증에는 적합하지만, 최종 UI 구조로 사용하면 다음 문제가 생긴다.

- 아트/레이아웃을 Unity Inspector에서 관리하기 어렵다.

- 매 프레임 문자열 생성과 UI layout 갱신이 발생한다.
- 일반/보스/정산 HUD가 하나의 prototype 클래스에 섞인다.
- 현재 `RunResult`를 받지만 실제 정산·저장은 수행하지 않는다.

협업 소유권 기준에서 전투 HUD/VFX는 Combat 영역에 둔다. 현재 `PrototypeRunHud`가 App에 있는 것은 Main 씬에서 빠르게 연결하기 위한 임시 배치이며, authored View/Presenter로 분리할 때 전투 HUD는 Combat 소유로 옮기고 App은 런 상태와 화면 라우팅만 연결한다.

다음 UI 단계에서는 `RunHudView`와 `RunHudPresenter`를 분리하고, 시간 표시와 카운터를 변경 시점 또는 낮은 주기로 갱신한다.

8. 정산과 Progression 경계

현재 Combat은 지갑을 직접 저장하지 않는다. `CombatRunController`가 보유하는 `GarnetEarned`는 이번 런에서 발생한 전투 사실이며, terminal 시 `RunResult`로 전달된다.

현재 흐름은 다음에서 멈춘다.

```
Combat facts
→ RunResult
→ HuntManager / PrototypeRunHud 결과 표시
→ 종료
```

아직 없는 것:

- `SettlementService`
- `SettlementReceipt`
- 영구 Wallet/Profile
- save schema migration
- 동일 `runId` 중복 지급 방지 저장소
- 저장 실패 재시도 큐
- 최초 보스 처치 해금
- 특성 구매 후 다음 런 스냅샷 반영

정산을 추가할 때는 `RunResult`를 받은 뒤에만 Progression이 움직이도록 한다. Combat/VFX/UI의 표시 수량을 영구 지갑의 지급 사실로 취급하지 않는다.

9. 테스트와 검증

현재 EditMode 테스트는 22개다.

런타임 계약 테스트

- 새 세션은 `Idle`에서 시작
- `Starting` → `Running` commit 분리
- Start 중복 거부
- `Complete/Abort` 상호 배타
- `Reset`은 `Completed` 이벤트를 발생시키지 않음
- terminal session 재시작 거부
- `RunId` 생성과 중복 claim 방지
- `RunRequest`의 ID/version/mode/boss/seed 검증
- `RunResult` 음수 누적값 거부

전투·스폰 테스트

- Spawner 시작 실패 시 Combat rollback
- 잘못된 SpawnPlan 결과 상태
- 다른 런의 대상 피해 거부
- 사망 즉시 unregister
- nested Animator와 Collider 없는 프리팹 어댑터
- authored monster prefab의 generic target adapter 연결
- 정확한 duration 경계에서 종료
- Pause 중 시계 정지
- 동일 seed의 동일 sequence
- 다른 seed의 다른 sequence

UI 테스트

- 초기 화면 이외의 root 비활성화
- 화면 전환 시 한 화면만 활성화
- 같은 화면을 반복 선택해도 중복 ScreenChanged 이벤트를 만들지 않음

검증 명령:

```
python3 tools/validate_collaboration.py
```

최근 격리 Unity 프로젝트 검증 결과:

- Unity EditMode: 22/22 passed
- Main 씬 import: 누락 스크립트 오류 없음
- 자동 PlayMode/build 검증: 아직 없음

Unity 테스트 결과는 협업 정적 검증과 별개로 보고한다. Unity가 실행되지 않는 환경에서는 validate_collaboration.py 통과만으로 컴파일 성공을 주장하지 않는다.

10. 현재 의도적으로 미구현인 것

다음은 누락이 아니라 이후 단계에서 붙일 확장점이다.

1. 여러 몬스터를 위한 ActiveTargetRegistry
2. MonsterCatalog, PrefabRegistry, RewardTableRegistry
3. SpawnPlan 다중 entry와 종별 생성 정책
4. Instantiate/Destroy 대신 풀링
5. 보스 HP, 파훼 표식, 성공 보너스, 실패 보상 0
6. 자동 공격과 스킬
7. Progression 구매·프로필·저장·복구
8. RunResult 뒤의 원자적 정산
9. 직렬화된 HUD View/Presenter와 UI 갱신 throttling
10. CombatSandbox/ProgressionSandbox 씬

현재 이 기능들을 위해 미리 지키는 경계는 다음과 같다.

- RunRequest와 RunResult는 GameObject를 모른다.

- `SpawnPlan`은 pure snapshot과 prefab binding을 분리한다.
- `RunId` token은 pooling 이후에도 stale callback 방어에 사용한다.
- `Combat`은 Progression 구현을 참조하지 않는다.
- 화면 전환은 Button event가 아니라 App router를 통한다.

11. 다음 구현 순서

Phase 1 - 현재 세로 슬라이스 안정화

- 프로토타입 duration을 실제 검증 목적에 맞게 명시하고 60초 acceptance test 추가
- HUD를 authored View와 Presenter로 분리
- `RunResult` 화면과 메인 메뉴 복귀 흐름 정리
- Main 씬의 직접 serialized reference를 점진적으로 명시화하고 `FindFirstObjectByType` 의존도 축소

Phase 2 - 콘텐츠 경계 일반화

- `MonsterCatalog`와 `PrefabRegistry` 추가
- `SpawnPlan` 여러 entry 소비
- `ICombatTarget` 조회를 `ActiveTargetRegistry`로 전환
- Walker 레거시 이름을 generic target adapter 이름으로 교체하거나 명시적 wrapper로 격리

Phase 3 - 성능과 대량 전투

- 대상 풀과 반환 lifecycle 도입
- active target registry의 현재 런 필터링
- overlap buffer와 대상 조회 할당량 측정
- UI 갱신 주기 분리
- 자동 공격 90 hit/s 기준 profiler acceptance test 추가

Phase 4 - Progression 연결

- `RunResult` → `SettlementService` → `SettlementReceipt` → `ProfileStore`
- `runId` 기반 idempotency와 저장 실패 재시도
- 특성 구매는 다음 런 snapshot에만 반영
- 영구 지갑과 런 중 누적 보상 표시 분리

Phase 5 - 보스와 전체 루프

- 보스 전용 snapshot과 파훼 입력
- 성공/실패/포기 정산 규칙
- 최초 처치 해금과 젬스톤 표시 규칙
- 엔딩과 엔딩 이후 자유 성장

12. 협업 시 지켜야 할 변경 규칙

코드 위치

작업	우선 위치
런 계약 변경	Assets/CursorHunter/Contracts/Runtime
정적 정의-검증	Assets/CursorHunter/Data
전투-스폰-대상-HUD/VFX	Assets/CursorHunter/Combat 또는 해당 Combat 어댑터
화면 전환-씬 조립-런 연결	Assets/CursorHunter/App
특성-재화-프로필-저장	Assets/CursorHunter/Progression
모듈별 테스트	각 모듈의 Tests

계약을 변경하면 생산자, 소비자, 테스트 더블을 같은 변경 단위에서 확인한다. `RunResult`에 필드를 추가하면서 Combat만 수정하거나, 저장 서비스만 먼저 만드는 식으로 한쪽만 갱신하지 않는다.

씬을 수정할 때는 `Main.unity`의 UI root와 Button persistent event를 먼저 확인한다. 화면 전환을 위해 새 Button에 직접 `GameObject.SetActive`를 연결하지 말고 `AppUiRootController`의 의미 있는 메서드를 추가한다.

새 몬스터를 추가할 때는 다음 체크리스트를 따른다.

1. 고정된 `monsterId`를 정한다.
2. `MonsterDefinition`을 만든다.
3. prefab 시각 계층과 Combat adapter를 확인한다.
4. collider가 공격 범위를 대표하는지 확인한다.
5. Hit/Death 상태가 없어도 안전하게 제거되는지 확인한다.
6. run token 검증과 unregister를 확인한다.
7. seed 재현·경계 시간·중복 이벤트 테스트를 추가한다.

13. 추가로 추천하는 문서

이 파일은 현재 구조의 기존 문서다. 프로젝트가 커지면 다음 문서를 별도로 두면 좋다.

문서	목적
CHANGELOG.md	버전별 실제 변경만 기록. 설계와 구현을 섞지 않음
ADR/0001-one-scene-ui-routing.md	One Scene과 App router를 선택한 이유·대안·폐기 조건 기록
contracts-current.md	현재 구현된 계약과 기획상 예정 계약을 분리
scene-binding-map.md	Main 씬 root, Button event, serialized fileID/GUID 변경 원장
test-matrix.md	런 상태·경계 시간·overflow·pool callback·정산 중복 테스트 표
asset-registry.md	monsterId/prefabKey/rewardTableId와 실제 에셋 매핑
performance-budget.md	최대 active target, spawn/s, attack/s, UI update/s, GC 목표
release-checklist.md	Unity 버전, 씬/빌드 목록, PlayMode, 저장 마이그레이션, 최종 에셋 점검

특히 `contracts-current.md`와 `contracts-planned.md`를 분리하는 것을 권장한다. 현재 존재하지 않는 `SettlementReceipt`나 `MonsterCatalog`를 이미 구현된 API처럼 문서에 적으면 협업자가 잘못된 참조를 만들기 쉽다.

14. 한 줄 요약

현재 프로토타입은 고유 런 계약과 rollback 가능한 일반 전투 시작, 현재 런 대상 보호, 중앙 UI 화면 라우팅까지 구현되어 있다. 다음 핵심 작업은 다중 콘텐츠 registry와 Progression 정산을 붙이기 전에 HUD와 target lifecycle을 확장 가능한 형태로 분리하는 것이다.